

Documentación del Proyecto spaCEinvaders

Instituto: Instituto Tecnológico de Costa Rica

Escuela: Ingeniería en Computación

Curso: CE3104 — Lenguajes, Compiladores e Intérpretes

Proyecto: spaCEinvaders — Implementación de Space Invaders con paradigmas imperativo y orientado a objetos

Desarrollador: Jestim Espinoza Morales

Carné: 2023282666

Tabla de contenidos

- [1.1 Introducción](#)
 - [1.2 Descripción de estructuras de datos](#)
 - [1.3 Descripción de algoritmos](#)
 - [1.4 Problemas sin solución](#)
 - [1.5 Plan de actividades](#)
 - [1.6 Problemas encontrados](#)
 - [1.7 Conclusiones](#)
 - [1.8 Recomendaciones](#)
 - [1.9 Bibliografía](#)
-

1.1 Introducción

spaCEinvaders es una implementación del clásico Space Invaders desarrollada para el curso CE3104 (Lenguajes, Compiladores e Intérpretes, TEC Costa Rica), con el objetivo de reafirmar los paradigmas de programación imperativo y orientado a objetos.

El proyecto está dividido en un servidor en Java, responsable de toda la lógica del juego (movimiento de aliens, colisiones, puntuación, oleadas) bajo un diseño orientado a objetos con patrones Abstract Factory, Observer y Adapter; y un cliente gráfico en C con Raylib, que se comunica con el servidor mediante sockets TCP y mensajes JSON, y se encarga únicamente de renderizar el estado recibido e enviar las acciones del jugador.

1.2 Descripción de estructuras de datos

1.2.1 Estructuras de datos en Java (servidor)

Lista de extraterrestres

```
List<Extraterrestre> extraterrestres; // ArrayList en EstadoJuego
```

Almacena los 55 aliens de una oleada (formación 5 filas × 11 columnas). Los aliens muertos permanecen en la lista con `vivo = false`; esto es intencional porque el frame-diff del cliente C necesita detectar la desaparición de un ID para emitir la explosión visual. El campo `id` es fijo y único por posición de la matriz: `id = 100 + fila * 11 + col`.

Campo	Tipo	Descripción
<code>id</code>	<code>int</code>	Identificador único fijo por posición de matriz
<code>x</code>	<code>double</code>	Posición horizontal en espacio lógico (0–100)
<code>y</code>	<code>double</code>	Posición vertical en espacio lógico (0–90)
<code>puntos</code>	<code>int</code>	Puntos que otorga al ser eliminado (10/20/40)
<code>tipo</code>	<code>String</code>	"CALAMAR", "CANGREJO" o "PULPO"
<code>vivo</code>	<code>boolean</code>	<code>false</code> si fue eliminado

Mapa de jugadores por sesión

```
Map<Integer, Jugador> jugadores; // HashMap en EstadoJuego
```

Permite acceso $O(1)$ al jugador de una sesión por su identificador de conexión. Cada entrada contiene:

Campo	Tipo	Descripción
<code>vidas</code>	<code>int</code>	Vidas restantes (inicia en 3)
<code>puntuacion</code>	<code>int</code>	Puntos acumulados
<code>posicionCanon</code>	<code>int</code>	Posición X del cañón (0–100)

Mapa de balas del jugador

```
Map<Integer, BalaJugador> balas; // ConcurrentHashMap en MotorJuego
```

Garantiza acceso seguro desde el hilo del `ScheduledExecutorService` (tick) y los hilos de red de cada `ManejadorCliente` que invocan `procesarMensajeCliente`. Cada `BalaJugador` contiene la posición (x, y), el estado activa y el método `mover()` que avanza 6 unidades hacia arriba por tick.

Lista de balas alienígenas

```
List<BalaAlien> balasAlien; // ArrayList en EstadoJuego
```

Cada `BalaAlien` contiene posición (x, y) y estado activa. Al inicio de cada tick se aplica `removeIf(b -> !b.isActiva())` para eliminar las balas que salieron del campo. El límite efectivo es de 16 balas simultáneas (definido en `MAX_BALAS_ALIEN`).

Mapas de sesiones múltiples

```
Map<Integer, MotorJuego> motores; // ConcurrentHashMap en ServidorJuego
Map<Integer, GameStateNotifier> notificadores; // ConcurrentHashMap en ServidorJuego
```

Implementan la arquitectura multi-sesión: cada cliente conectado tiene su propio motor de juego completamente aislado. Al desconectarse un jugador se destruye únicamente su entrada, sin afectar las partidas de otros jugadores.

Lista de observadores

```
CopyOnWriteArrayList<GameObserver> observadores; // en GameStateNotifier
```

CopyOnWriteArrayList permite que el hilo del executor itere los observadores para notificarlos sin necesidad de un lock explícito, mientras que los hilos de red pueden añadir o eliminar observadores concurrentemente sin causar ConcurrentModificationException.

1.2.2 Estructuras de datos en C (cliente)

HitBox

```
typedef struct {
    float x, y, width, height;
} HitBox;
```

Layout idéntico al Rectangle de Raylib, lo que permite pasarla directamente a CheckCollisionRecs(). Se recalcula cada frame a partir de las coordenadas lógicas del juego.

Extraterrestre

```
typedef struct {
    int id;
    float x, y;
    int puntos;
    int tipo; // TIPO_CALAMAR=0, TIPO_CANGREJO=1, TIPO_PULPO=2
    int vivo;
    int golpe_visual; // 1 = parpadeo de impacto (hit feedback predictivo)
    HitBox hitbox;
} Extraterrestre;
```

55 slots estáticos en EstadoJuego. El campo golpe_visual se activa en el cliente cuando CheckCollisionRecs() detecta colisión con la bala, antes de recibir confirmación del servidor (feedback predictivo).

EstadoJuego (struct central del cliente)

```
typedef struct {
    Jugador jugador;
    Extraterrestre extraterrestres[MAX_EXTRATERR]; // 55 slots
```

```

    int            cantidad_extraterrestres;
    Ovni           ovni;
    Bunker         bunkers[NUM_BUNKERS];           // 4 bunkers
    BalaAlien      balas_alien[MAX_BALAS_ALIEN];    // 16 slots
    int            cantidad_balas_alien;
    int            bala_activa;
    float          bala_x, bala_y;
    int            fin_juego;
    int            puntuacion_final;
    int            ronda;
    float          velocidad;
} EstadoJuego;

```

Este struct se copia íntegramente (snapshot de ~2.5 KB) bajo mutex en cada frame, permitiendo que el hilo de render trabaje sobre una copia segura mientras el hilo de red puede actualizar el original libremente.

Pool de partículas

```

typedef struct {
    float x, y;    // posición en píxeles de pantalla
    float vx, vy;  // velocidad en píxeles/segundo
    float vida;    // segundos restantes (0 = slot libre)
    Color col;     // color con canal alpha
} Particula;

```

```
static Particula s_pool[MAX_PARTICULAS]; // MAX_PARTICULAS = 128
```

Pool de tamaño fijo en memoria estática. Elimina completamente el uso de malloc/free en el game loop. Los slots se reutilizan cuando vida <= 0.0f.

AlienSlot (frame-diff de efectos)

```

typedef struct {
    int vivo;
    int px, py; // posición en píxeles (última conocida)
    Color col;  // color del alien (para colorear la explosión)
} AlienSlot;

```

```
static AlienSlot s_alien_prev[55]; // indexado por id - 100
```

Mantiene el estado del frame anterior para detectar aliens que desaparecen del snapshot del servidor (lo que indica que murieron). Al detectar un ID que estaba vivo y ya no aparece, se emite una explosión de partículas en la última posición conocida.

BunkerVisual (State Pattern)

```

typedef struct {
    uint8_t pixeles[BV_FILAS][BV_COLS]; // grilla 12x18 de celdas vivas/muertas
    int      salud_anterior;
    int      inicializado;
}

```

```
} BunkerVisual;
```

```
BunkerVisual g_bunkers_visual[NUM_BUNKERS]; // 4 bunkers globales
```

Cada celda es un bloque de BV_PX_SIZE × BV_PX_SIZE píxeles. La función bv_sincronizar() aplica el patrón State: destruye celdas desde los bordes (erosión) cuando el porcentaje baja, y restaura la forma original cuando el porcentaje sube (nueva ronda).

1.3 Descripción de algoritmos

1.3.1 Game loop del servidor

El servidor utiliza ScheduledExecutorService.scheduleAtFixedRate con un período de 50 ms (20 fps lógicos). La función tick() es synchronized para garantizar exclusión mutua con los hilos de red. El orden fijo de operaciones por tick es:

1. moverExtraterrestres() – avanza la formación y gestiona rebotes
2. moverBalasJugador() – mueve cada bala activa 6 unidades hacia arriba
3. moverBalasAlien() – mueve cada bala alien 6 unidades hacia abajo
4. disparoAlienAleatorio() – selecciona tirador y dispara si han pasado ≥1500ms
5. actualizarOvni() – mueve OVNI o lo spawea si han pasado ≥20s
6. verificarColisiones() – bala vs alien, bala vs OVNI, bala alien vs jugador, bala alien vs bunker, alien vs bunker
7. verificarCondicionFin() – ronda completa, jugador sin vidas, alien llega al suelo
8. notificarEstado() – serializar y enviar ESTADO_JUEGO a todos los clientes

1.3.2 Movimiento de la formación con acumulador fraccional

El movimiento de los aliens utiliza un acumulador para permitir velocidades fraccionarias sin aritmética de punto fijo:

```
acumuladorAlien += velocidadActual;
if (acumuladorAlien >= 1.0) {
    int pasos = (int) acumuladorAlien;
    acumuladorAlien -= pasos;
    // mover cada alien 'pasos' unidades en la dirección actual
}
```

Con velocidadActual = 0.5, los aliens avanzan 1 unidad cada 2 ticks, es decir, a 10 unidades/segundo. La velocidad escala un 20% al completar cada oleada: velocidadActual *= 1.2.

1.3.3 Rebote de la formación con clamp

Al detectar que el alien más a la derecha superó ANCHO - MARGEN o el más a la izquierda bajó de MARGEN:

```
if (maxX >= ANCHO - 4 || minX <= 4) {
    direccion *= -1;
    for (Extraterrestre e : obtenerVivos()) {
        e.establecerX(Math.max(4, Math.min(ANCHO - 4, e.obtenerX())));
        e.establecerY(e.obtenerY() + pasoDescenso);
    }
    acumuladorAlien = 0.0;
    velocidadActual *= FACTOR_DESCENSO; // escala adicional por descenso
}
```

El clamp [4, ANCHO-4] previene que un alien creado por la consola de administración en una posición extrema cause rebote en cada tick subsiguiente.

1.3.4 Selección de tiradores por columna

Para seleccionar qué alien dispara en cada oportunidad, se agrupan los aliens vivos por columna lógica y se selecciona el de mayor Y (más cercano al jugador):

```
Map<Integer, Extraterrestre> tiradores = new HashMap<>();
for (Extraterrestre e : obtenerVivos()) {
    int col = (int)(e.obtenerX() / 8);
    if (!tiradores.containsKey(col) ||
        e.obtenerY() > tiradores.get(col).obtenerY()) {
        tiradores.put(col, e);
    }
}
List<Extraterrestre> lista = new ArrayList<>(tiradores.values());
// selecciona uno aleatoriamente y crea BalaAlien en su posición
```

El disparo se inhibe si ya existe una bala alien activa desde el mismo tirador. El intervalo mínimo entre disparos es de 1500 ms (controlado por tiempoUltimoDisparoAlien).

1.3.5 Detección de colisiones con distancia de Chebyshev

Se utiliza la distancia de Chebyshev (máximo de diferencias absolutas en X e Y) para la detección de colisiones bala-alien y bala-OVNI:

```
private boolean colisionaBala(double bx, double by, double tx, double ty)
{
    return Math.max(Math.abs(bx - tx), Math.abs(by - ty)) <= RADIO_COLISION;
}
// RADIO_COLISION = 2 → caja de detección 4x4 unidades (≈35x25 px)
```

La distancia de Chebyshev es computacionalmente más barata que la distancia euclidiana (evita la raíz cuadrada) y produce una caja de colisión alineada a los ejes, adecuada para sprites rectangulares.

1.3.6 Ring buffer TCP en el cliente C

El parser de red acumula datos TCP en un buffer circular para manejar la fragmentación de paquetes:

```
static char rbuf[8192];
static int  rbuf_end = 0;

int n = recv(sockfd, rbuf + rbuf_end, sizeof(rbuf) - rbuf_end - 1, 0);
rbuf_end += n;
rbuf[rbuf_end] = '\0';

char *nl = strchr(rbuf, '\n');
while (nl) {
    *nl = '\0';
    // procesar mensaje completo en rbuf
    int len = (int)(nl - rbuf) + 1;
    memmove(rbuf, nl + 1, rbuf_end - len);
    rbuf_end -= len;
    nl = strchr(rbuf, '\n');
}
```

La lectura en bloques de 512 bytes reduce los syscalls de red en aproximadamente 250 veces comparado con la lectura byte a byte.

1.3.7 Parser JSON manual en C

Se implementó un parser JSON mínimo sin dependencias externas. Las funciones principales son:

```
int  json_int(const char *json, const char *key);    // extrae entero
float json_float(const char *json, const char *key); // extrae flotante
int  json_str(const char *json, const char *key,
              char *out, int maxlen);               // extrae cadena
int  json_int_array(const char *json, const char *key,
                   int *out, int maxn);             // extrae array de
enteros
```

La función `json_int` maneja explícitamente los booleanos JSON (`true` → 1, `false` → 0), ya que `balaActiva` se serializa como booleano desde Java.

1.3.8 Patrón snapshot para render seguro con hilo de red

El bucle principal del cliente opera con dos hilos: el hilo de render (principal) y el hilo de red. El problema fundamental es que `EndDrawing()` espera la sincronización

vertical (VSync, ~16 ms en 60 Hz), tiempo durante el cual el hilo de render mantendría el mutex si el render se hiciera directamente sobre el estado compartido:

```
// Incorrecto: mutex tomado 16ms → hilo de red bloqueado
pthread_mutex_lock(&mtx);
render_frame(&estado); // EndDrawing() espera VSync aquí
pthread_mutex_unlock(&mtx);

// Correcto: mutex tomado ~1µs → hilo de red libre durante render
pthread_mutex_lock(&mtx);
EstadoJuego snap = estado; // copia de struct (~2.5KB, < 1µs)
pthread_mutex_unlock(&mtx);
render_frame(&snap); // VSync sin mutex
```

1.3.9 Frame-diff para detección de muertes de aliens

El protocolo del servidor envía únicamente los aliens vivos en cada snapshot. Para detectar qué alien murió entre dos frames (y emitir la explosión), se mantiene el estado del frame anterior:

```
void efectos_sincronizar_alienos(const Extraterrestre *alienos, int n,
                                const int *px_arr, const int *py_arr) {
    int visto[55] = {0};
    for (int i = 0; i < n; i++) {
        int idx = alienos[i].id - 100;
        visto[idx] = 1;
        s_alienos_prev[idx] = (AlienSlot){1, px_arr[i], py_arr[i], color_p
or_tipo(alienos[i].tipo)};
    }
    for (int idx = 0; idx < 55; idx++) {
        if (s_alienos_prev[idx].vivo && !visto[idx]) {
            // alien desapareció → emitir explosión en última posición co
nocida
            efectos_emitir_explosion(s_alienos_prev[idx].px,
                                    s_alienos_prev[idx].py,
                                    s_alienos_prev[idx].col);
            s_alienos_prev[idx].vivo = 0;
        }
    }
}
```

1.3.10 Strategy Pattern para sprites de aliens

En lugar de un switch en el loop de render, se utiliza una tabla de punteros a función:

```
typedef void (*AlienRenderFn)(int cx, int cy, float escala, Color col);

static AlienRenderFn s_renderers[3] = {
    sprites_dibujar_calamar,
    sprites_dibujar_cangrejo,
```



```

        sprites_dibujar_pulpo
    };

void sprites_dibujar_alien(int tipo, int cx, int cy, float escala, Color
col) {
    if (tipo >= 0 && tipo < 3)
        s_renders[tipo](cx, cy, escala, col);
}

```

Cada función de dibujo utiliza un helper `draw_pixel_grid()` que renderiza una grilla de bloques rectangulares de 3×3 píxeles (con 1 píxel de separación entre bloques) para producir el aspecto pixel-art retro del Space Invaders original.

1.3.11 Animación de dos fotogramas

Los sprites de aliens alternan entre dos poses para simular movimiento, replicando el comportamiento del arcade original:

```

static int s_anim_frame = 0; // 0 o 1
static int s_anim_ticks = 0;

void sprites_canon_tick(void) {
    s_anim_ticks++;
    if (s_anim_ticks >= 20) { // cada ~0.33s a 60fps
        s_anim_frame = 1 - s_anim_frame;
        s_anim_ticks = 0;
    }
}

```

Cada función de sprite selecciona `rows_a[]` o `rows_b[]` según el valor de `s_anim_frame`.

1.3.12 Mapeo de coordenadas lógicas a píxeles

El espacio lógico del juego es 100×90 unidades (definido en el servidor Java). El cliente mapea estas coordenadas al área de render de la ventana:

```

static inline int gcol(float gx) {
    return 10 + (int)(gx * 880.0f / 100.0f);
}
static inline int grow(float gy) {
    return 55 + (int)(gy * 570.0f / 90.0f);
}

```

El área de juego ocupa de `x=10` a `x=890` y de `y=55` a `y=625` en la ventana de 900×680 píxeles.

1.3.13 Bypass del focus-click de Windows mediante GetAsyncKeyState

Windows convierte el primer clic después de que una ventana obtiene el foco en un mensaje WM_ACTIVATE, que GLFW procesa como activación de ventana en lugar de WM_LBUTTONDOWN. Raylib nunca recibe ese evento como pulsación de botón. La solución es leer el estado del hardware directamente:

```
#ifdef _WIN32
extern __declspec(dllimport) short __stdcall GetAsyncKeyState(int vKey);

static int mouse_click_detectar(void) {
    static int prev = 0;
    int cur = (GetAsyncKeyState(0x01) & 0x8000) ? 1 : 0;
    int click = cur && !prev;
    prev = cur;
    return click;
}
#else
static int mouse_click_detectar(void) {
    return IsMouseButtonPressed(MOUSE_LEFT_BUTTON);
}
#endif
```

GetAsyncKeyState invoca directamente NtUserGetAsyncKeyState en el kernel Windows a través de user32.dll, sin pasar por la cola de mensajes Win32 ni por GLFW. El detector de flanco (prev) garantiza que se registre exactamente una pulsación por clic.

1.3.14 Erosión de bunkers desde los bordes

La función `bv_destruir_patron_borde()` destruye las celdas más alejadas del centro del bunker para simular daño acumulado desde los extremos:

```
// 1. Calcular distancia de cada celda viva al centro
// 2. Ordenar por distancia descendente (burbuja sobre ≤216 elementos)
// 3. Destruir las primeras 'a_destruir' celdas con jitter aleatorio
// 4. Actualizar grilla de píxeles
```

El jitter aleatorio (± 0.8 en la distancia) previene que la destrucción sea perfectamente simétrica, produciendo un aspecto más orgánico similar al juego original.

1.4 Problemas sin solución

1.4.1 Controlador físico con Raspberry Pi Pico (Parte 4 — Anexo)

Descripción del requerimiento:

El enunciado de la tarea solicita, la implementación de un controlador físico usando una placa Raspberry Pi Pico. El controlador debía conectarse al computador mediante UART (o I2C) y exponer al menos dos botones físicos: uno para mover el cañón y otro para disparar.

1.5 Plan de actividades

1.5.1 Registro de sesiones de desarrollo

El proyecto fue desarrollado individualmente por **Jestim Espinoza Morales** en cinco sesiones entre el 5 y el 13 de junio de 2026 (≈32 horas efectivas).

Fecha	Actividades realizadas	Duración estimada	Responsable
2026-06-05	Arquitectura general, modelos de dominio Java y patrones Abstract Factory/Observer/Adapter, protocolo JSON/TCP y pruebas unitarias iniciales.	≈ 4 horas	Jestim Espinoza Morales
2026-06-06	MotorJuego completo (game loop, movimiento, colisiones, OVNI), ServidorJuego multi-sesión, ConsolaAdmin y cliente C base.	≈ 8 horas	Jestim Espinoza Morales
2026-06-07	Flujo Observador/Reiniciar, refactor de AdaptadorMensaje, oleadas automáticas de aliens y mejoras de física y de ConsolaAdmin.	≈ 6 horas	Jestim Espinoza Morales
2026-06-12	Sprites pixel-art, sistema de partículas, destrucción visual de bunkers y corrección de catorce problemas de estabilidad/foco de clic.	≈ 10 horas	Jestim Espinoza Morales
2026-06-13	Rediseño visual retro arcade (HUD, CRT, paleta fosforescente), animación	≈ 4 horas	Jestim Espinoza Morales

Fecha	Actividades realizadas	Duración estimada	Responsable
	de dos fotogramas y traducción de la interfaz.		

1.6 Problemas encontrados

A continuación se documentan los doce problemas más significativos encontrados durante el desarrollo. Cada entrada describe el síntoma observado, la causa raíz identificada, los intentos de solución que no funcionaron (cuando los hubo), la solución definitiva y la lección que puede generalizarse a proyectos similares.

Problema 1: Bala del jugador invisible (2026-06-06)

Descripción: Al disparar, la bala nunca aparecía en pantalla a pesar de que el servidor reportaba el estado `balaActiva: true` en sus mensajes.

Causa raíz: La función `render_frame()` incluye internamente `EndDrawing()`, que espera la sincronización vertical del monitor (VSync, aproximadamente 16 ms a 60 Hz). Esta espera se realizaba mientras el mutex del estado del juego estaba tomado. El hilo de red (`hilo_red`) quedaba así bloqueado durante 16 ms en cada frame, equivalente al 99% del tiempo disponible entre frames. En ese intervalo de bloqueo, los mensajes `balaActiva: true` (bala activa) y `balaActiva: false` (bala destruida en el mismo tick del servidor) llegaban juntos en el mismo segmento TCP. Al recuperar el mutex, el hilo de red aplicaba ambos mensajes consecutivamente y el estado que el render veía siempre era `false`.

Intentos fallidos: - Aumentar `INTERVALO_MS` del servidor de 50 ms a 200 ms: empeoró el problema, ya que el servidor tardaba más en generar la transición `true` → `false` y los mensajes seguían llegando juntos. - Ajustar `VELOCIDAD_BALA` entre 2 y 8 unidades por tick: el problema no era de velocidad sino de sincronización entre hilos.

Solución implementada: Patrón snapshot — se copia el struct `EstadoJuego` (~2.5 KB) bajo mutex en aproximadamente 1 µs, y se llama `render_frame()` sobre la copia, fuera del mutex. La espera de VSync (~16 ms) no bloquea al hilo de red.

```
/* Copia del estado bajo mutex (~1 µs) */
pthread_mutex_lock(&mtx);
EstadoJuego snap = estado;
pthread_mutex_unlock(&mtx);
/* Render desde la copia, sin mutex */
render_frame(&snap);
```

Lección aprendida: En aplicaciones que combinan un hilo de red con un hilo de render, nunca mantener un mutex de estado durante operaciones que involucren sincronización con el hardware de video. El tiempo de bloqueo de VSync es varios órdenes de magnitud mayor que el tiempo necesario para copiar el estado del juego.

Problema 2: Mensajes de disparo se perdían — bala no salía (2026-06-06)

Descripción: Al presionar la barra espaciadora, la bala no aparecía, o aparecía con un retraso de varios segundos. El comportamiento era intermitente.

Causa raíz: El movimiento del cañón se implementó inicialmente con `IsKeyDown`, que devuelve verdadero en cada frame mientras la tecla permanece presionada. A 60 fps, esto generaba hasta 60 mensajes `MOVER_CANON` por segundo. El buffer TCP del servidor se saturaba con mensajes de movimiento y los mensajes `DISPARO` llegaban tarde al servidor o eran descartados.

Intentos fallidos: Añadir un throttle de 50 ms entre envíos consecutivos de `MOVER_CANON`.

Solución implementada: Reemplazar `IsKeyDown` por `IsKeyPressed` para la detección del movimiento. `IsKeyPressed` devuelve verdadero únicamente en el frame exacto en que se presiona la tecla, generando exactamente 1 mensaje por pulsación física y eliminando la saturación del buffer TCP.

Lección aprendida: En protocolos de red para videojuegos, es fundamental distinguir entre input de estado continuo (aceleración, giro mantenido) e input de evento discreto (disparo, salto). `IsKeyDown` es adecuado para lógica local de alta frecuencia; `IsKeyPressed` es apropiado cuando cada pulsación debe generar exactamente un mensaje de red.

Problema 3: El segundo cliente destruía la partida del primer cliente (2026-06-06)

Descripción: Al conectarse un segundo jugador, el juego del primero se detenía o terminaba inesperadamente de inmediato.

Causa raíz: La implementación original utilizaba un único `MotorJuego` global compartido por todas las conexiones. Al desconectarse cualquier cliente — incluso el último en conectarse — el método `desconectarJugador()` invocaba `motorGlobal.detener()`, deteniendo el único motor existente y destruyendo las partidas de todos los jugadores activos.

Solución implementada: Refactorización de `ServidorJuego` para mantener un `Map<Integer, MotorJuego>` indexado por el ID de conexión de cada cliente. Cada conexión nueva crea su propio triplete independiente compuesto por `EstadoJuego`,

GameStateNotifier y MotorJuego. La desconexión de un cliente detiene y elimina únicamente su propia entrada del mapa, sin afectar las sesiones de otros jugadores.

Lección aprendida: Los recursos con estado de sesión deben estar aislados por cliente. Un motor de juego global es un anti-patrón en servidores multi-cliente, ya que el ciclo de vida de un recurso compartido queda supeditado al comportamiento de cualquier cliente individual.

Problema 4: Servidor lento y errático con dos clientes conectados (2026-06-06)

Descripción: Con dos jugadores conectados simultáneamente, el juego presentaba movimientos con retraso pronunciado y mensajes que llegaban en orden incorrecto.

Causa raíz: El método `alCambiarEstadoJuego()` de `ManejadorCliente`, invocado desde el bloque `synchronized` del executor 20 veces por segundo por cada sesión activa, contenía una llamada a `System.out.printf` que imprimía el JSON completo del estado (~600 caracteres) en cada notificación. `PrintStream` en Java es bloqueante; con dos sesiones activas en paralelo se producía contención en la salida estándar, retrasando la liberación del monitor del executor y acumulando ticks del game loop.

Intentos fallidos: Reducir la longitud del mensaje de log al incluir solo campos seleccionados.

Solución implementada: Eliminar todos los logs del tipo `ESTADO_JUEGO` dentro del bloque sincronizado. Se conservan únicamente los logs de eventos de baja frecuencia: conexión, desconexión y fin de juego.

Lección aprendida: El logging de alta frecuencia dentro de secciones críticas es una fuente común de degradación de rendimiento en sistemas concurrentes. Los logs en el camino caliente del game loop deben evitarse o implementarse mediante buffers asíncronos.

1.7 Conclusiones

- **Objetivo general cumplido:** desarrollar el servidor Java (OO) y el cliente C (imperativo) en paralelo, conectados por un protocolo JSON/TCP, reafirmó en la práctica ambos paradigmas de programación.
- **Objetivo específico 1 (C y Java):** se construyeron aplicaciones completas en ambos lenguajes (~15 clases Java con tests, 12 módulos C), evidenciando diferencias en manejo de memoria, tipado y concurrencia.
- **Objetivo específico 2 (imperativo vs. OO):** los patrones Abstract Factory, Observer y Adapter organizaron la complejidad en Java; en C, soluciones

imperativas directas (ring buffer, parser JSON manual, pool de partículas) priorizaron control y rendimiento.

- **Objetivo específico 3 (listas):** ArrayList/CopyOnWriteArrayList en Java y arrays estáticos de tamaño fijo en C resolvieron el mismo problema —gestión de colecciones— según las restricciones de cada paradigma (GC vs. loop de alta frecuencia).
 - **Sincronización multihilo:** los bugs más difíciles (bala invisible, servidor degradado, frame-diff de muertes) mostraron que la visibilidad de datos entre hilos y el alcance de los locks son críticos para rendimiento y corrección.
 - **Limitaciones del estado único de verdad:** simplifica el cliente, pero introduce latencia mínima de un tick (50 ms), compensada con hitboxes predictivas y frame-diff.
 - **Ventajas del paradigma imperativo:** pools estáticos, estructuras de tamaño fijo y ausencia de efectos secundarios ocultos en C permitieron un control de rendimiento no alcanzable con un lenguaje de alto nivel con GC.
-

1.8 Recomendaciones

- **Protocolo con eventos de cambio:** sustituir los snapshots completos cada 50 ms por delta encoding reduciría el ancho de banda (~80%) y eliminaría la necesidad del frame-diff de partículas.
 - **Reconexión automática en el cliente:** implementar backoff exponencial (1s, 2s, 4s, 8s...) ante cortes de red, en vez de terminar con un mensaje de error.
 - **Compresión del payload JSON:** aplicar LZ4 sobre los mensajes ESTADO_JUEGO (300-600 bytes c/u) reduciría el ancho de banda en 50-70% con múltiples clientes.
 - **Implementar la Parte 4 (Raspberry Pi Pico):** como un tercer hilo en el cliente C que lea el puerto serie y reutilice los mismos mensajes que el teclado.
 - **Pruebas de integración de red para el cliente C:** un servidor de prueba mínimo que emita secuencias ESTADO_JUEGO ayudaría a detectar temprano bugs de parser/render.
 - **Externalizar la configuración de red:** mover IP y puertos de constants.h a un archivo de configuración o argumentos de línea de comandos.
 - **Serialización binaria:** migrar de JSON a un formato binario compacto (p. ej. Protocol Buffers) reduciría la carga del parser y la latencia cuando la red sea el cuello de botella.
-

1.9 Bibliografía

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. (Patrones Abstract Factory, Observer, Adapter)
2. **Tanenbaum, A. S., & Wetherall, D. J.** (2011). *Computer Networks* (5th ed.). Prentice Hall. (Protocolo TCP, algoritmo de Nagle, TCP_NODELAY)
3. **Ramsey, R.** (2021). *Raylib documentation and source code*. <https://www.raylib.com> — Biblioteca de gráficos 2D/3D utilizada para el cliente C.
4. **Microsoft Corporation.** (2024). *GetAsyncKeyState function (winuser.h)*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getasynckeystate>
5. **Oracle Corporation.** (2024). *Java SE 11 API Specification: ScheduledExecutorService*. <https://docs.oracle.com/en/java/docs/api/java.base/java/util/concurrent/ScheduledExecutorService.html>
6. **Oracle Corporation.** (2024). *Java SE 11 API Specification: CopyOnWriteArrayList*. <https://docs.oracle.com/en/java/docs/api/java.base/java/util/concurrent/CopyOnWriteArrayList.html>
7. **Taito Corporation.** (1978). *Space Invaders* [Videojuego arcade]. (Referencia de diseño original para la mecánica del juego, la formación de aliens, el OVNI y los bunkers)
8. **Stevens, W. R., Fenner, B., & Rudoff, A. M.** (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley. (Implementación del ring buffer TCP y manejo de fragmentación de paquetes)
9. **Instituto Tecnológico de Costa Rica.** (2026). *Enunciado Tarea 4 — CE3104 Lenguajes, Compiladores e Intérpretes* [Documento PDF]. Escuela de Ingeniería en Computación.
10. **Kernighan, B. W., & Ritchie, D. M.** (1988). *The C Programming Language* (2nd ed.). Prentice Hall. (Referencia del lenguaje C utilizado en el cliente)
11. **Eckel, B.** (2006). *Thinking in Java* (4th ed.). Prentice Hall. (Referencia de programación orientada a objetos en Java)

12. **Pressman, R. S., & Maxim, B. R.** (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education. (Fundamentos de documentación de software y plan de actividades)

2.0 GitHub

<https://github.com/JestimEM/spaCEinvaders.git>